

Données : $\begin{cases} A, \text{ une matrice triangulaire inférieure} \\ \text{de taille } n \times n \text{ avec } \det A \neq 0 \\ b, \text{ un vecteur de taille } n \times 1 \end{cases}$
Résultat : x , la solution du système $Ax = b$

1.1 pour $i \leftarrow 1$ **à** n **faire**

1.2 $\left| \begin{array}{l} x_i^* = \frac{b_i - \sum_{j=1}^{i-1} a_{ij} x_j^*}{a_{ii}}; \end{array} \right.$

1.3 finpour

/ Coût : n^2 */*

/ Avantages :*

- Facile à implanter ;*
- Peu d'erreurs d'arrondi*

**/*

Algorithme 1 : Phase descendante d'élimination

Données : $\begin{cases} A, \text{ une matrice triangulaire supérieure} \\ \text{de taille } n \times n \text{ avec } \det A \neq 0 \\ b, \text{ un vecteur de taille } n \times 1 \end{cases}$
Résultat : x , la solution du système $Ax = b$

2.1 pour $i \leftarrow n$ **à 1 faire**

2.2 $\left| \begin{array}{l} x_i^* = \frac{b_i - \sum_{j=i+1}^n a_{ij}x_j^*}{a_{ii}}; \end{array} \right.$

2.3 finpour

/* Coût : n^2 */

/* Avantages :

- Facile à implanter ;
- Peu d'erreurs d'arrondi

***/**

Algorithme 2 : Phase ascendante d'élimination

Données : $\begin{cases} A, \text{ une matrice de taille } n \times n \text{ avec } \det A \neq 0 \\ b, \text{ un vecteur de taille } n \times 1 \end{cases}$

Résultat : x , la solution du système $Ax = b$

3.1 Initialisation : $A^{(0)} \leftarrow A$ et $b^{(0)} \leftarrow b$

3.2 pour $k \leftarrow 1$ **à** $n - 1$ **faire**

3.3 **si** $a_{kk}^{(k-1)} = 0$ **alors**

3.4 | trouver une ligne i avec $i > k$ et $a_{ik}^{(k-1)} \neq 0$; /* existe, car $\det A \neq 0$ */

3.5 | permuter les lignes k et i ;

3.6 | $A^{(k-1)} \leftarrow$ la matrice permutée;

3.7 | $b^{(k-1)} \leftarrow$ le vecteur b permuté de la même manière;

3.8 **pour** $i \leftarrow k + 1$ **à** n **faire**

3.9 | **pour** $j \leftarrow k + 1$ **à** n **faire**

3.10 | | $m_{ik} \leftarrow \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}} ;$

3.11 | | $a_{ij}^{(k)} \leftarrow a_{ij}^{(k-1)} - m_{ik}a_{kj}^{(k-1)} ;$

3.12 | | $b_i^{(k)} \leftarrow b_i^{(k-1)} - m_{ik}b_k^{(k-1)} ;$

3.13 Faire une phase ascendante d'élimination avec $A^{(n-1)}$ et $b^{(n-1)}$.

/* **Coût :** $\frac{2}{3}n^3$ si aucune permutation */

/* **Avantages :**

- Plus simple que la méthode du déterminant ou de Cramer ;
- Calculs de A^{-1} et de $\det A$ possibles

*/

Algorithme 3 : Méthode de Gauss

Données : A , une matrice de taille $n \times n$

Résultat : La valeur propre la plus près de $\lambda_1^{(0)}$ et le vecteur propre qui lui est associé

4.1 **Initialisation :** Choisir z_0 tel que $\|z_0\|_2 = 1$

4.2 **pour** $k = 0, 1, \dots$ **faire**

4.3 $\lambda_1^{(k)} \leftarrow z_k^t A z_k$;

4.4 Résoudre $(A - \lambda_1^{(k)} \mathbb{1})y = z_k$;

4.5 $z_{k+1} \leftarrow y / \|y\|_2$;

/* **Remarque :** $\|z_k - x_1\|_2 \leq \alpha \|z_{k-1} - x_1\|_2^3$ pour une constante $\alpha \in \mathbb{R}$ */

/* **Désavantages :** À chaque itération, il faut refaire une décomposition LU */

Algorithme 4 : Méthode du quotient de Rayleigh

Données : A , une matrice de taille $n \times n$

Résultat : Toutes les valeurs propres de A et les vecteurs propres qui lui sont associés

5.1 Initialisation : $i = 1$ et $A^{(i)} = A$

5.2 Trouver λ_1 et x_1 à l'aide d'une méthode appropriée ;

5.3 pour $i < n$ **faire**

5.4 $u_i \leftarrow x_i + \|x_i\|_2 \cdot (\text{signe}((x_i)_1), 0, 0, \dots, 0)^t$;

5.5 $P_i \leftarrow \mathbb{I} - \frac{u_i u_i^t}{\|x_i\|_2 \cdot (\|x_i\|_2 + |(x_i)_1|)}$;

5.6 Calculer $P_i A^{(i)} P_i^{-1}$;

5.7 Extraire de $P_i A^{(i)} P_i^{-1}$ la sous-matrice $A^{(i+1)}$ en biffant la 1^{re} ligne et la 1^{re} colonne ;

5.8 Trouver λ_{i+1} , un estimé de la valeur propre dominante de $A^{(i+1)}$;

5.9 Appliquer la méthode des puissances inverses à la matrice $A - \lambda_{i+1} \mathbb{I}$, mettre à jour la valeur de λ_{i+1} et déterminer x_{i+1} ;

5.10 $i \leftarrow i + 1$;

/ Avantages : Cette méthode permet de déterminer les multiplicités des valeurs propres de la matrice A .*

**/*

Algorithme 5 : Méthode d'élimination et de réduction